

PurpleForce Gym

Sistema de Gestión Integral de Gimnasio

MEMORIA DEL PROYECTO INTERMODULAR

Autor: Daniel García Docío

Ciclo / Curso: 2º DAM — 2025/26

Centro: Nortempo Formación

Fecha de entrega: 8 de junio de 2026

Exposición: 12 de junio de 2026

Tecnologías principales

Java 17 · Spring Boot 3.2 · Spring Security 6 · MySQL 8 · Thymeleaf · OpenPDF · JUnit 5

Índice

Resumen (Abstract)	3
1. Motivación e introducción	4
1.1 Motivación	4
1.2 Contexto y problema a resolver	4
1.3 Objetivos propuestos	4
2. Metodología, recursos y planificación	5
2.1 Metodología utilizada	5
2.2 Estimación de recursos	5
2.3 Planificación (hitos y cronograma)	5
3. Tecnologías y herramientas	6
4. Análisis	7
4.1 Definición de requisitos	7
4.2 Modelo de datos	8
4.3 Casos de uso	9
5. Diseño	10
5.1 Diseño general / arquitectura	10
5.2 Diagrama de clases (UML) y ERD	10
5.3 Capturas de interfaz y navegación	11
5.4 Decisiones de diseño importantes	12
6. Implementación	13
6.1 Estructura del proyecto	13
6.2 Componentes principales	13
6.3 Dificultades encontradas y soluciones	14
7. Multiusuario concurrente	15
8. Despliegue e instalación	17
8.1 Manual de instalación	17
8.2 Configuración	17
8.3 Carga de datos de ejemplo	17
8.4 Manual de usuario con capturas	18
9. Pruebas	20
9.1 Plan de pruebas	20
9.2 Tabla resumen de pruebas	20
9.3 Evidencias de pruebas	21
10. Resultados, conclusiones y vías futuras	27
11. Glosario	29

12. Bibliografía y webgrafía	30
Anexos	31
Anexo A — Capturas adicionales	31
Anexo B — SQL de ejemplo	31
Anexo C — Configuración	32

Resumen (Abstract)

PurpleForce Gym es una aplicación web completa para la gestión integral de un gimnasio. Está dirigida a tres tipos de usuarios: administradores, instructores y socios. El sistema centraliza la consulta y reserva de clases, la gestión de entrenadores y tipos de actividad, el control de aforo en tiempo real y la generación de informes exportables en PDF y CSV.

Las funcionalidades clave son: autenticación segura con tres roles diferenciados (ADMIN, INSTRUCTOR, SOCIO), reserva de clases con control de concurrencia transaccional, ranking de clases más populares basado en consultas JPQL avanzadas, exportación de informes PDF personalizados y gestión completa del catálogo de ejercicios y entrenadores.

1. Motivación e introducción

1.1 Motivación

La gestión de un gimnasio implica coordinar horarios, entrenadores, aforos y reservas de manera simultánea. En muchos centros pequeños y medianos esta tarea se realiza con hojas de cálculo, llamadas telefónicas o aplicaciones genéricas que no cubren las necesidades específicas del sector. PurpleForce Gym nace con el objetivo de proporcionar una solución web integral, accesible desde cualquier dispositivo, que automatice y centralice toda la operativa del gimnasio.

1.2 Contexto y problema a resolver

Antes de la aplicación, un socio debía llamar al gimnasio para reservar una clase, sin saber en tiempo real cuántas plazas quedaban disponibles. El administrador llevaba un registro manual de las reservas, con el riesgo de sobre-reservas y errores humanos. Los instructores no disponían de forma digital de consultar su agenda ni la lista de alumnos inscritos.

PurpleForce Gym resuelve todos estos problemas: el socio reserva desde la web en segundos, el sistema controla el aforo evitando sobre-reservas mediante transacciones en base de datos, y tanto el administrador como el instructor disponen de un panel centralizado con toda la información actualizada.

1.3 Objetivos propuestos

Objetivo general:

- Desarrollar una aplicación web funcional y completa que cubra los requisitos técnicos del módulo Proyecto Intermodular de 2º DAM.

Objetivos específicos:

- Implementar autenticación segura con tres roles de usuario distintos (ADMIN, INSTRUCTOR, SOCIO).
- Gestionar el ciclo completo de reservas con control de aforo en tiempo real.

- Desarrollar consultas JPQL no triviales: ranking de popularidad e historial por usuario.
- Generar informes exportables en PDF (historial de socio, horario de instructor) y CSV (ranking).
- Garantizar el manejo concurrente de usuarios mediante transacciones y sesiones independientes.
- Documentar el proyecto con pruebas funcionales y memoria técnica completa.

2. Metodología, recursos y planificación

2.1 Metodología utilizada

Se ha seguido una metodología iterativa e incremental, dividiendo el proyecto en sprints semanales. Se empezó por la capa de persistencia y seguridad, añadiendo funcionalidades de manera progresiva. Se realizaron commits frecuentes en GitHub y se utilizó Postman para probar los endpoints manualmente durante el desarrollo.

2.2 Estimación de recursos

Hardware:

- PC de desarrollo: procesador Intel Core i5, 16 GB RAM, SSD 512 GB.
- Servidor local: la aplicación se ejecuta en localhost:8080.

Software:

- Visual Studio Code con Extension Pack for Java y Spring Boot Extension Pack.
- MySQL 8.0 + MySQL Workbench — base de datos relacional y cliente gráfico.
- Git + GitHub — control de versiones y repositorio remoto.
- Postman — pruebas manuales de endpoints HTTP.
- Maven 3.8 — gestión de dependencias y construcción del proyecto.

2.3 Planificación (hitos y cronograma)

Semana / Fechas	Hito	Tareas principales
1-2 (09-20 abr)	Análisis y planificación	Requisitos, modelo de datos, diagrama ER
3-4 (21 abr-04 may)	Configuración y entidades	Spring Boot, entidades JPA, SecurityConfig
5-6 (05-18 may)	Backend principal	Repositorios, servicios, controladores Admin y Auth
7 (19-25 may)	Panel Socio e Instructor	Reservas, filtros, historial, panel instructor
8 (26 may-01 jun)	Informes y exportaciones	PDF con OpenPDF y CSV con Apache Commons

9 (02-08 jun)	Pruebas y correcciones	10 pruebas JUnit, corrección de bugs
10 (09-12 jun)	Memoria y presentación	Redacción memoria, preparar demo

3. Tecnologías y herramientas

Tecnología	Categoría	Uso en el proyecto
Java 17	Lenguaje	Backend y lógica de negocio completa
Spring Boot 3.2	Framework	Servidor, rutas e inyección de dependencias
Spring Security 6	Seguridad	Autenticación, roles, BCrypt y sesiones HTTP
Spring Data JPA	Persistencia	Repositorios JPA con consultas JPQL avanzadas
Thymeleaf 3.1	Motor plantillas	Renderizado HTML en servidor con Spring MVC
MySQL 8.0	Base de datos	BD relacional para todos los datos del sistema
H2 (tests)	BD tests	Base de datos en memoria para ejecución de JUnit
Bootstrap Icons	UI / iconos	Librería de iconos SVG para la interfaz web
OpenPDF 1.3	Informes PDF	Exportación de historial y horarios en PDF
Apache Commons CSV	Exportación	Generación del ranking de clases en CSV
Lombok	Código	Generación automática de getters, setters y builders
JUnit 5 + MockMvc	Testing	Pruebas funcionales con contexto Spring completo
Maven 3.8	Build	Gestión del proyecto y sus dependencias
VS Code	IDE	Entorno de desarrollo principal
Git + GitHub	Versiones	Seguimiento de cambios y repositorio remoto

4. Análisis

4.1 Definición de requisitos

Requisitos funcionales

ID	Nombre	Descripción
RF-01	Login con roles	Autenticación con tres roles: ADMIN, INSTRUCTOR, SOCIO
RF-02	Registro de socios	Un nuevo usuario puede crear su cuenta como socio

RF-03	Gestión de clases	El admin puede crear, editar y cancelar clases
RF-04	Gestión entrenadores	El admin puede crear, editar y desactivar entrenadores
RF-05	Tipos de clase	El admin define tipos con precio, nivel y duración
RF-06	Catálogo ejercicios	El admin gestiona el catálogo completo de ejercicios
RF-07	Reservar clase	Un socio reserva plaza en una clase con disponibilidad
RF-08	Cancelar reserva	Un socio cancela una reserva confirmada previamente
RF-09	Historial de reservas	El socio ve su historial completo con estado
RF-10	Ranking de clases	Clases ordenadas por reservas (consulta no trivial 1)
RF-11	Historial por usuario	Reservas filtradas y ordenadas por usuario (consulta no trivial 2)
RF-12	Exportar PDF	El socio descarga su historial PDF; instructor su horario PDF
RF-13	Exportar CSV	El admin exporta el ranking de clases en formato CSV
RF-14	Ver alumnos	El instructor ve los socios inscritos en cada clase suya
RF-15	Filtros de búsqueda	Las clases se filtran por nivel y fecha combinados

Requisitos no funcionales

ID	Categoría	Descripción
RNF-01	Seguridad	Contraseñas cifradas con BCrypt; roles protegidos con Spring Security
RNF-02	Rendimiento	Respuesta inferior a 1 segundo en operaciones CRUD normales
RNF-03	Usabilidad	Interfaz responsiva con mensajes de error comprensibles
RNF-04	Portabilidad	Ejecutable en local con Java 17 y MySQL 8
RNF-05	Mantenibilidad	Código organizado en capas MVC con comentarios y pruebas
RNF-06	Concurrencia	Soporte de sesiones múltiples simultáneas; transacciones ACID

4.2 Modelo de datos

La aplicación utiliza una base de datos relacional MySQL con seis entidades principales. Las relaciones más importantes son: **Clase** tiene una relación N:1 con TipoClase y N:1 con Entrenador. **Reserva** tiene relaciones N:1 con Clase y N:1 con Usuario, formando la tabla de intersección central del sistema de reservas.

Entidad	Atributos principales	Descripción
usuario	id, nombre, apellidos, email, password, rol, activo, fechaAlta	Persona con acceso al sistema

entrenador	id, nombre, apellidos, especialidad, telefono, email, activo	Imparte las clases del gimnasio
tipo_clase	id, nombre, nivel, duracionMinutos, precio, color	Modalidad de clase (Yoga, HIIT...)
clase	id, tipo_clase_id, entrenador_id, fecha, horalnicio, horaFin, aforoMaximo, sala, activa	Sesión concreta en día y hora
ejercicio	id, nombre, grupoMuscular, dificultad, materialNecesario	Ejercicio del catálogo del gimnasio
reserva	id, clase_id, usuario_id, fechaReserva, estado (CONFIRMADA/CANCELADA)	Inscripción de socio a una clase

Justificación de MySQL: la elección de MySQL como sistema de gestión de base de datos para PurpleForce Gym se justifica por los siguientes motivos:

- **Datos relacionales:** los datos del gimnasio son altamente relacionales. Una reserva pertenece a un usuario y a una clase concreta, y una clase tiene asignado un entrenador y un tipo. Las relaciones N:1 se modelan con naturalidad en un esquema relacional.
- **Soporte transaccional:** MySQL garantiza transacciones ACID, lo que es crítico para el control de concurrencia en las reservas. Cuando dos socios intentan reservar la última plaza simultáneamente, la transacción garantiza que solo una tenga éxito.
- **Integridad referencial:** las claves foráneas impiden que existan reservas sin clase o usuario válidos, y que se borren clases que tienen reservas asociadas, manteniendo la coherencia de los datos en todo momento.
- **Escalabilidad:** MySQL es capaz de gestionar miles de usuarios y reservas simultáneas sin degradación del rendimiento, lo que lo hace adecuado para un gimnasio en crecimiento.
- **Compatibilidad con Spring Data JPA:** MySQL tiene integración nativa con Hibernate y Spring Data JPA, lo que permite definir repositorios con CRUD automático y escribir consultas JPQL avanzadas de forma limpia y mantenible.
- **Facilidad de mantenimiento:** MySQL es gratuito, ampliamente conocido en el sector y cuenta con herramientas visuales como MySQL Workbench que facilitan la administración, las copias de seguridad y el diagnóstico de problemas.
- **Madurez y fiabilidad:** con más de 25 años de desarrollo activo, MySQL es uno de los sistemas de base de datos más probados del mundo, lo que garantiza su estabilidad en entornos de producción.

4.3 Casos de uso

ID	Actor	Nombre	Descripción
CU-01	SOCIO	Registrarse	El socio crea su cuenta con email y contraseña
CU-02	TODOS	Iniciar sesión	El usuario se autentica y accede a su panel
CU-03	SOCIO	Ver catálogo	El socio visualiza clases con filtros por nivel y fecha
CU-04	SOCIO	Reservar clase	El socio reserva plaza en clase con disponibilidad
CU-05	SOCIO	Cancelar reserva	El socio cancela una reserva confirmada
CU-06	SOCIO	Ver historial	El socio consulta su historial completo de reservas
CU-07	SOCIO	Descargar PDF	El socio exporta su historial en formato PDF
CU-08	SOCIO	Ver ranking	El socio consulta las clases más populares
CU-09	INSTRUCTOR	Ver mis clases	El instructor consulta sus clases asignadas
CU-10	INSTRUCTOR	Ver alumnos	El instructor ve los socios inscritos en cada clase
CU-11	ADMIN	Gestionar clases	El admin crea, edita y cancela sesiones de clase
CU-12	ADMIN	Gestionar entrenadores	El admin da de alta, edita y desactiva entrenadores
CU-13	ADMIN	Ver informes	El admin consulta estadísticas y rankings avanzados
CU-14	ADMIN	Exportar CSV	El admin descarga el ranking de clases en CSV

5. Diseño

5.1 Diseño general / arquitectura

La aplicación sigue una arquitectura cliente-servidor en tres capas implementada mediante el patrón MVC de Spring Boot. La capa de presentación está formada por plantillas Thymeleaf con HTML/CSS. La capa de negocio la componen los controladores Spring MVC que delegan en los servicios. La capa de persistencia usa repositorios Spring Data JPA mapeados a tablas MySQL.

El flujo completo de una petición es: Navegador → Dispatcher Servlet → Controller → Service → Repository → MySQL → Thymeleaf → HTML al navegador.

5.2 Diagrama de clases (UML)

Las clases principales del dominio, sus atributos clave y sus relaciones se muestran en el siguiente diagrama:

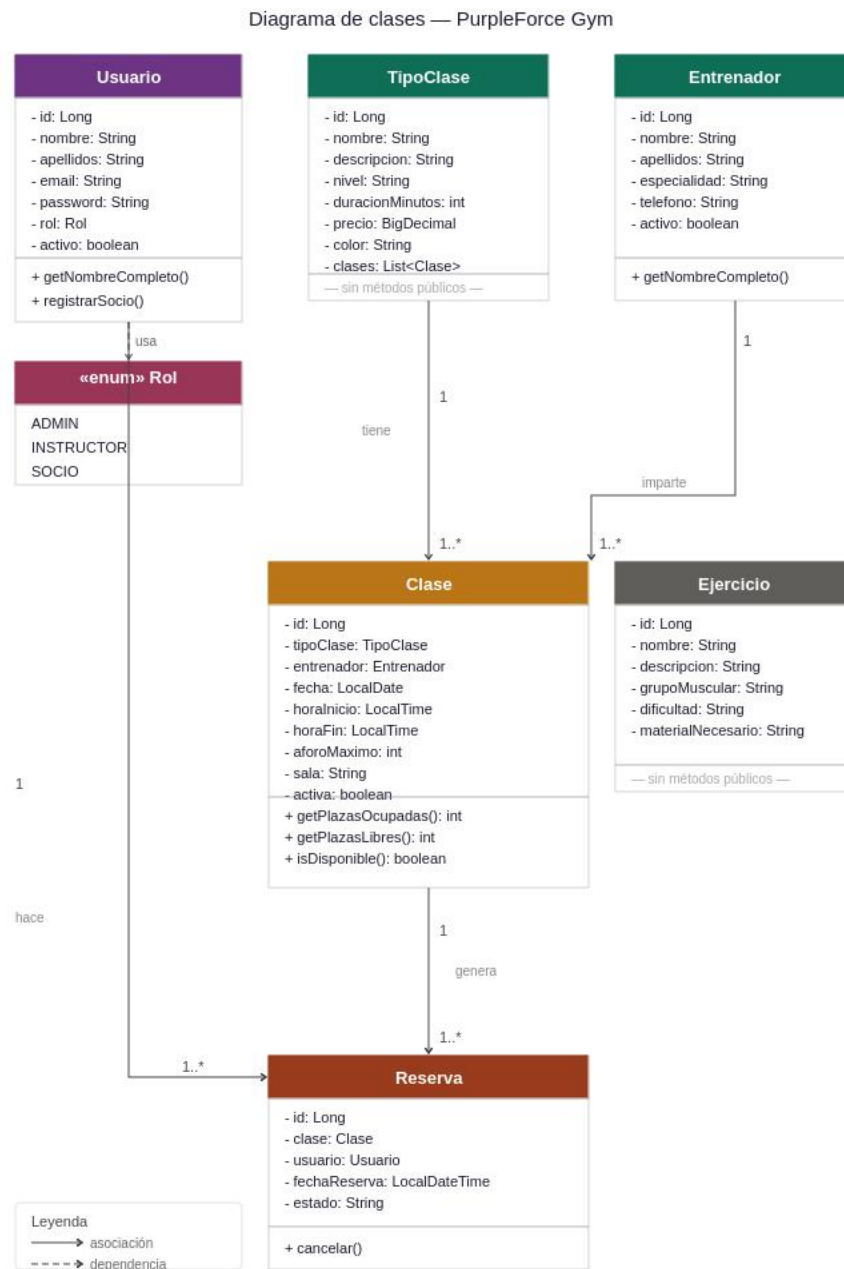


Figura UML. Diagrama de clases de PurpleForce Gym con atributos, métodos y relaciones.

5.2b Diagrama Entidad-Relación (ERD)

El siguiente diagrama muestra las relaciones entre las tablas de la base de datos de PurpleForce Gym, con sus atributos, claves primarias (PK), claves foráneas (FK) y claves únicas (UK).

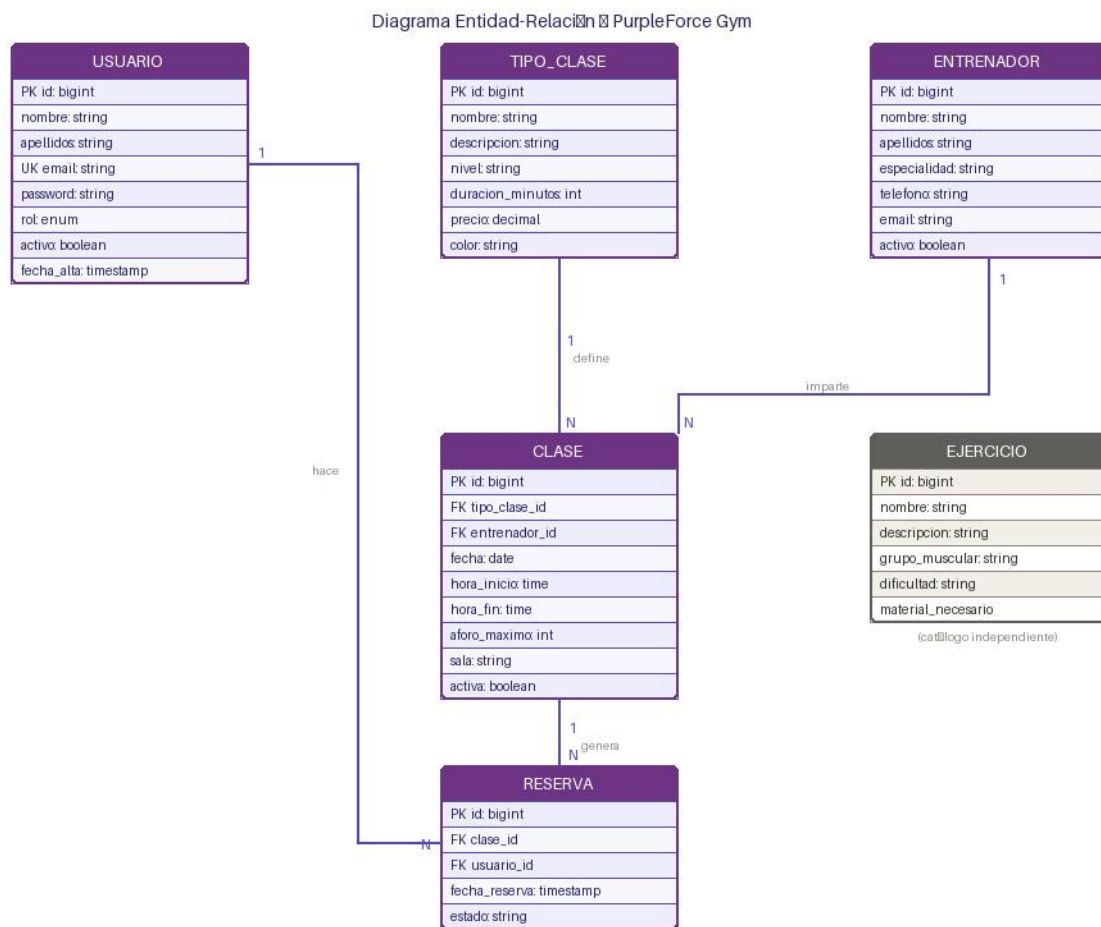


Figura ERD. Diagrama Entidad-Relación de PurpleForce Gym con atributos, claves y relaciones entre tablas.

La entidad central del sistema es **Reserva**, ya que conecta usuarios y clases. **Clase** se relaciona con TipoClase (que define el tipo de actividad, nivel y precio) y con Entrenador (quien la imparte), mientras que **Usuario** representa los distintos roles del sistema (ADMIN, INSTRUCTOR y SOCIO). La entidad **Ejercicio** es independiente y actúa como catálogo de actividades físicas disponibles en el gimnasio. Todas las relaciones siguen el patrón N:1, lo que permite mantener la integridad referencial y simplificar las consultas JPQL.

Descripción textual de las clases principales:

- **Usuario** (id, nombre, apellidos, email, password, rol: Rol, activo, fechaAlta) → 1:N Reserva
- **Entrenador** (id, nombre, apellidos, especialidad, telefono, email, activo) → 1:N Clase
- **TipoClase** (id, nombre, descripcion, nivel, duracionMinutos, precio, color) → 1:N Clase
- **Clase** (id, tipoClase, entrenador, fecha, horaInicio, horaFin, aforoMaximo, sala, activa) → 1:N Reserva. Métodos: getPlazasOcupadas(), getPlazasLibres(), isDisponible()
- **Reserva** (id, clase, usuario, fechaReserva, estado) — tabla de intersección central

- **Ejercicio** (id, nombre, descripcion, grupoMuscular, dificultad, materialNecesario) — entidad independiente
- **Enum Rol:** ADMIN, INSTRUCTOR, SOCIO

Relaciones: Clase N:1 TipoClase, Clase N:1 Entrenador, Reserva N:1 Clase, Reserva N:1 Usuario.

5.3 Capturas de interfaz y navegación

El flujo de navegación principal: Login → redirige al dashboard según el rol del usuario.

- **Admin:** Dashboard → Clases → Tipos → Entrenadores → Ejercicios → Usuarios → Informes
- **Socio:** Dashboard → Catálogo de clases (con filtros) → Mis Reservas → Ranking
- **Instructor:** Dashboard → Mis Clases → Lista de alumnos → Descargar horario PDF

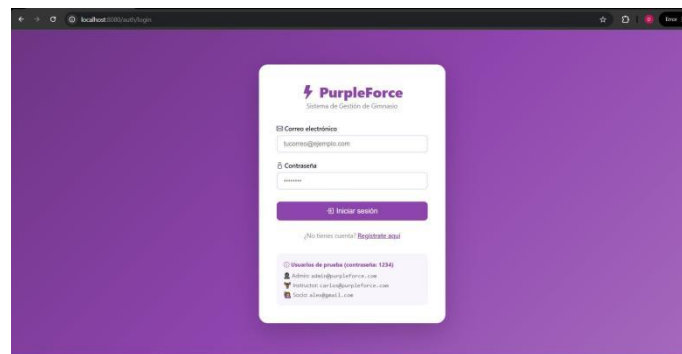


Figura 1. Pantalla de login con fondo morado y usuarios de prueba.

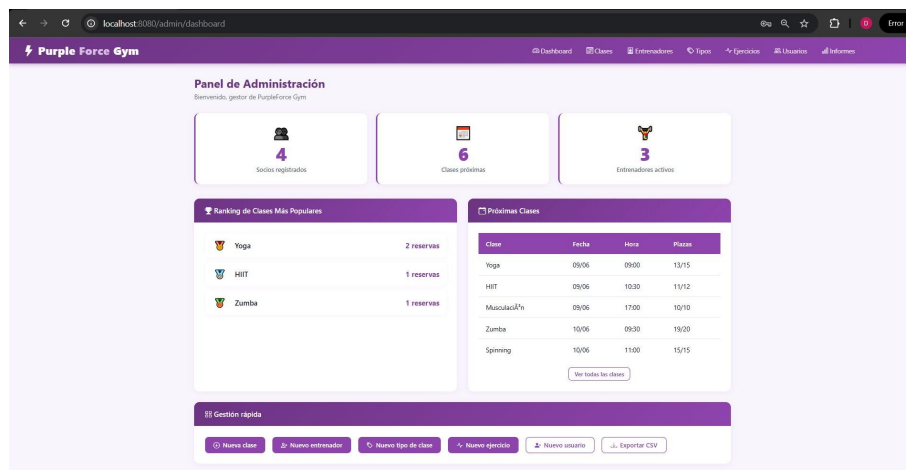


Figura 2. Panel de administración con estadísticas, ranking y próximas clases.

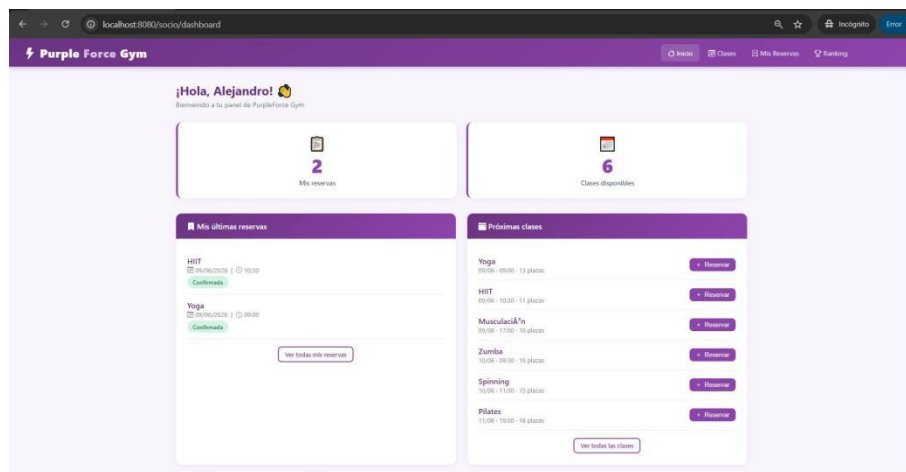


Figura 3. Panel del socio con últimas reservas y clases disponibles.

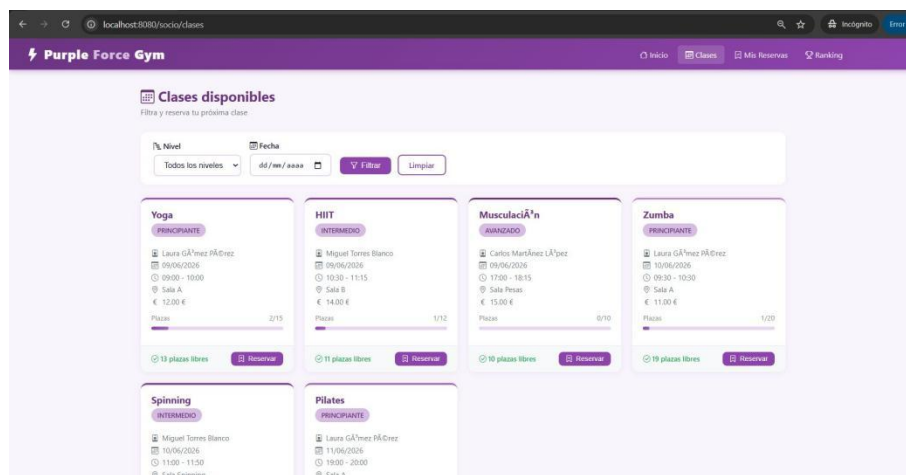


Figura 4. Catálogo de clases con filtros y barras de aforo.

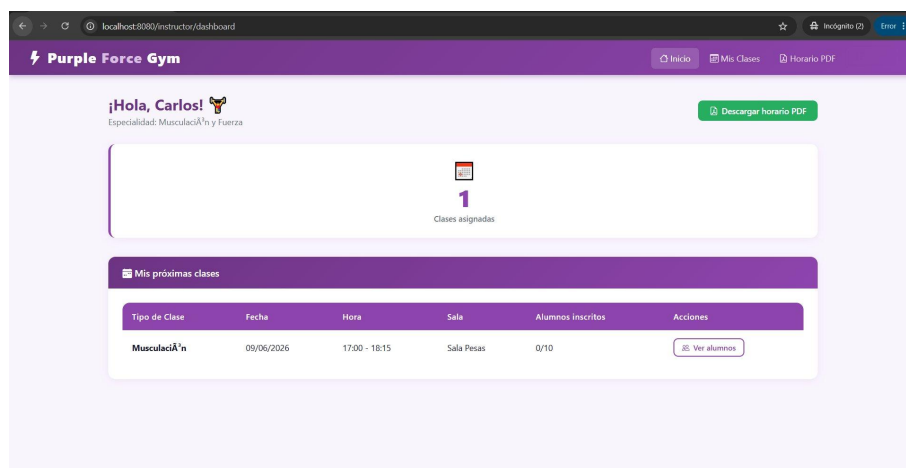


Figura 5. Panel del instructor con clases asignadas.

5.4 Decisiones de diseño importantes

- 1. **Control de concurrencia:** `@Transactional` en `ReservaService.reservar()` garantiza operaciones atómicas.

- **2. PasswordConfig separado:** se extrajo PasswordEncoder a clase propia para evitar referencia circular.
- **3. Redirección por rol:** el successHandler redirige automáticamente al dashboard correcto según el rol.
- **4. data.sql idempotente:** todos los INSERT usan WHERE NOT EXISTS para no duplicar datos en reinicios.

6. Implementación

6.1 Estructura del proyecto

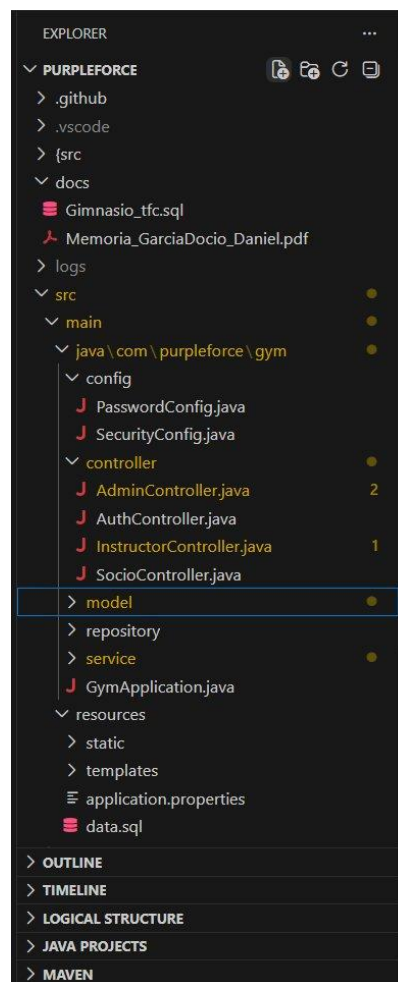


Figura. Estructura del proyecto en VS Code: config, controller, model, repository y service.

- **com.purpleforce.gym.model** — Entidades JPA: Usuario, Entrenador, TipoClase, Clase, Ejercicio, Reserva, Rol

- `com.purpleforce.gym.repository` — Interfaces `JpaRepository` con consultas `@Query JPQL`
- `com.purpleforce.gym.service` — Lógica de negocio: `UsuarioService`, `ClaseService`, `ReservaService`, `InformeService`
- `com.purpleforce.gym.controller` — `AdminController`, `SocioController`, `InstructorController`, `AuthController`
- `com.purpleforce.gym.config` — `SecurityConfig` (rutas y login) + `PasswordConfig` (BCrypt)
- `src/main/resources/templates/` — Plantillas Thymeleaf en `admin/`, `socio/`, `instructor/` y `auth/`
- `src/main/resources/data.sql` — Datos de ejemplo cargados automáticamente al arrancar

6.2 Componentes principales

Capa de persistencia

Los repositorios extienden `JpaRepository` proporcionando CRUD automático. Se definen consultas JPQL con `@Query` para operaciones no triviales. El ranking de clases se calcula con un COUNT agrupado por tipo de clase, ordenado descendientemente. El historial de usuario filtra reservas CONFIRMADAS y las ordena por fecha descendente.

Capa de servicios y concurrencia

`ReservaService` implementa el control de concurrencia mediante `@Transactional`. El método `reservar()` recarga la clase desde la BD, cuenta las plazas y solo si hay disponibilidad crea la reserva. `InformeService` genera PDF con OpenPDF y CSV con Apache Commons CSVPrinter.

Seguridad

`SecurityConfig` configura Spring Security 6 con autenticación por formulario. Las rutas `/admin/**` requieren `ROLE_ADMIN`, `/instructor/**` requieren `ROLE_INSTRUCTOR` o `ROLE_ADMIN`, y `/socio/**` requieren `ROLE_SOCIO` o `ROLE_ADMIN`. Las contraseñas se almacenan con `BCryptPasswordEncoder` con factor 10.

6.3 Dificultades encontradas y soluciones

- **Problema 1:** referencia circular `SecurityConfig` ↔ `UsuarioService`. **Solución:** extraer `PasswordEncoder` a clase `PasswordConfig` separada.
- **Problema 2:** `data.sql` fallaba en reinicios por claves duplicadas. **Solución:** todos los INSERT usan `WHERE NOT EXISTS`.
- **Problema 3:** Spring Security 6 cambió la API. **Solución:** migrar al nuevo flujo lambda-based eliminando `WebSecurityConfigurerAdapter`.
- **Problema 4:** tabla clase con estructura diferente al importar el SQL original. **Solución:** usar `ddl-auto=create` para que Spring Boot cree las tablas limpias.

7. Multiusuario concurrente

7.1 Identificación de usuarios

La autenticación se realiza mediante formulario web gestionado por Spring Security. Al hacer login, Spring crea una sesión HTTP independiente para cada usuario con un JSESSIONID en cookie. Cada sesión mantiene el objeto Authentication con email y rol, sin posibilidad de mezcla entre sesiones distintas.

7.2 Demostración de dos usuarios simultáneos

La captura siguiente muestra dos sesiones activas simultáneamente: el administrador en el panel de informes (navegador normal) y el socio Alejandro en el ranking de clases (ventana de incógnito). Ambas sesiones son completamente independientes.

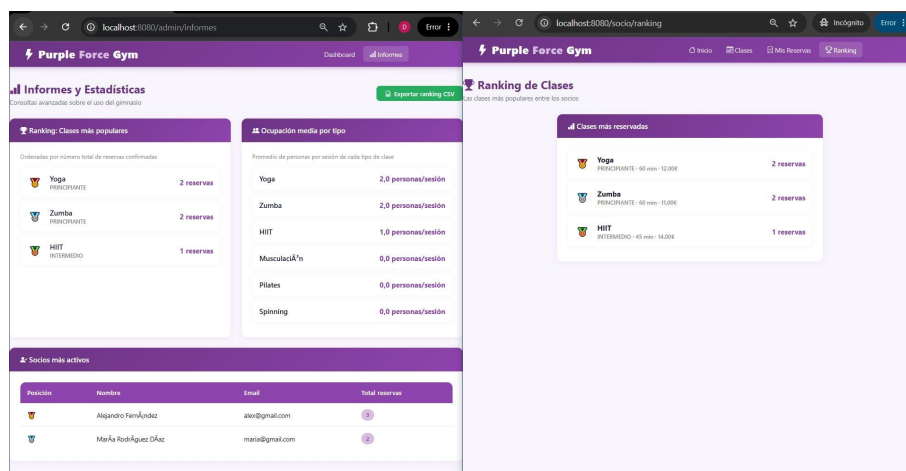


Figura 6. Dos sesiones activas simultáneamente: Admin (izquierda) y Socio (derecha).

7.3 Control de concurrencia en reservas

ReservaService.reservar() está anotado con @Transactional, garantizando que la lectura del aforo y la creación de la reserva sean atómicas. MySQL aplica aislamiento REPEATABLE READ, asegurando que solo un socio obtenga la plaza si dos intentan reservar simultáneamente.

7.4 Fallos de comunicación

Si el servidor no está disponible, Spring Boot devuelve una página de error HTTP estándar. Los formularios incluyen tokens CSRF. Si la sesión expira, Spring Security redirige automáticamente al login.

8. Despliegue e instalación

8.1 Manual de instalación

Requisitos previos:

- Java 17 o superior instalado (JDK, no solo JRE)
- Maven 3.8 o superior
- MySQL 8.0 en ejecución local
- Visual Studio Code con Extension Pack for Java y Spring Boot Extension Pack

Pasos de instalación:

1. Clonar el repositorio: `git clone https://github.com/dagardoc/PurpleForce.git`
2. Crear la BD: `CREATE DATABASE gimnasio_db CHARACTER SET utf8mb4;`
3. Configurar credenciales en `src/main/resources/application.properties`
4. Abrir `GymApplication.java` en VS Code y pulsar el botón Run
5. Acceder en el navegador a `http://localhost:8080`

8.2 Configuración

El fichero `application.properties` contiene toda la configuración. Las credenciales de MySQL se configuran en `spring.datasource.username` y `spring.datasource.password`. No se suben contraseñas reales al repositorio.

8.3 Carga de datos de ejemplo

Los datos de ejemplo se cargan automáticamente al arrancar mediante `data.sql`. Las sentencias usan `WHERE NOT EXISTS` para no duplicar datos en reinicios.

8.4 Manual de usuario con capturas

Como Administrador (`admin@purpleforce.com` / `1234`): gestionar clases, entrenadores, tipos, ejercicios y usuarios. El apartado Informes muestra estadísticas y permite exportar el ranking en CSV.

Tipo de Clase	Entrenador	Fecha	Hora	Sala	Aforo	Nivel	Estado	Acciones
Yoga	Laura Gázquez Pérez	09/06/2026	09:00 - 10:00	Sala A	2/15	PRINCIANTE	Activa	 
HIIT	Miguel Torres Blanco	09/06/2026	10:30 - 11:15	Sala B	1/12	INTERMEDIO	Activa	 
Musculación	Carlos Martínez López	09/06/2026	17:00 - 18:15	Sala Pesas	0/10	AVANZADO	Activa	 
Zumba	Laura Gázquez Pérez	10/06/2026	09:30 - 10:30	Sala A	1/20	PRINCIANTE	Activa	 
Spinning	Miguel Torres Blanco	10/06/2026	11:00 - 11:50	Sala Spinning	0/15	INTERMEDIO	Activa	 
Pilates	Laura Gázquez Pérez	11/06/2026	19:00 - 20:00	Sala A	0/18	PRINCIANTE	Activa	 

Figura 7. Gestión de clases: el administrador puede crear, editar y cancelar sesiones.

Como Socio (alex@gmail.com / 1234): explorar el catálogo, reservar clases, consultar el historial y descargarlo en PDF.

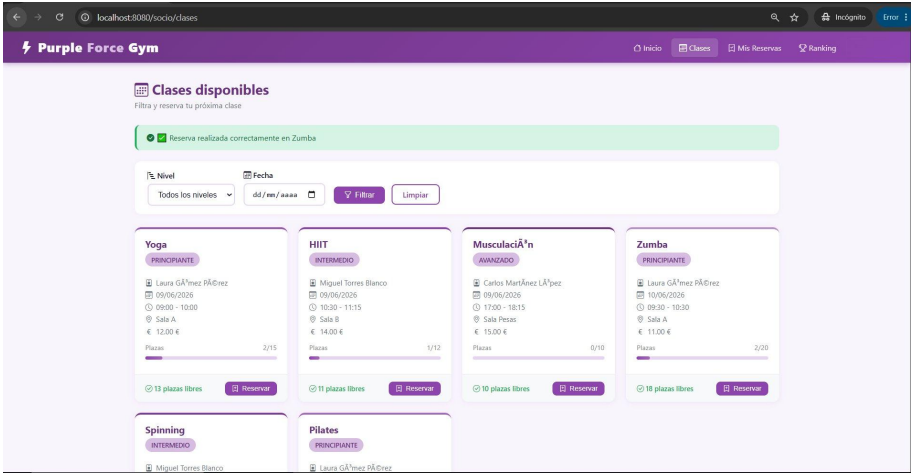


Figura 8. Confirmación de reserva con mensaje verde tras reservar correctamente.

Como Instructor (carlos@purpleforce.com / 1234): ver las clases asignadas, consultar los alumnos inscritos y descargar el horario en PDF.

9. Pruebas

9.1 Plan de pruebas

El objetivo es verificar que todas las funcionalidades principales responden correctamente tanto en el camino feliz (datos válidos) como en casos de error (credenciales incorrectas, reservas duplicadas, acceso no autorizado). Se han realizado pruebas manuales con capturas de evidencia y pruebas automáticas con JUnit 5.

9.2 Tabla resumen de pruebas

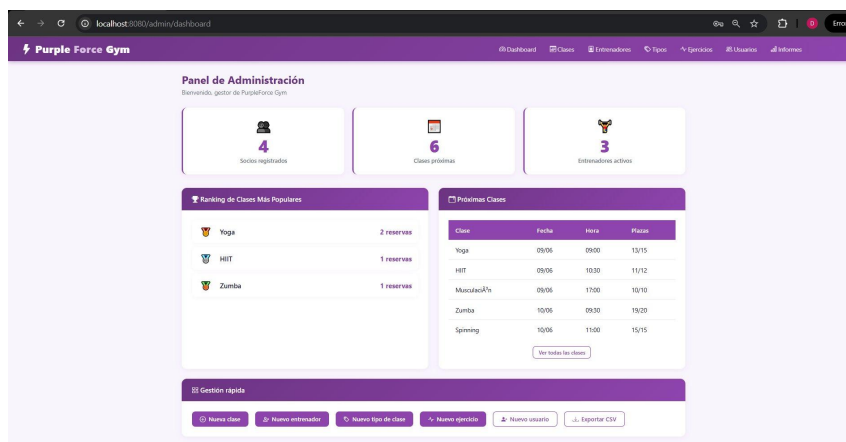
ID	Prueba	Resultado esperado	Resultado obtenido	OK
PT-01	Login con credenciales correctas	Redirección al dashboard de admin	Redirección correcta al panel admin	✓
PT-02	Login con credenciales incorrectas	Mensaje de error en rojo	Mensaje 'Email o contraseña incorrectos'	✓
PT-03	Reservar clase con plazas disponibles	Reserva con estado CONFIRMADA	Mensaje verde y reserva creada	✓

PT-04	Cancelar reserva confirmada	Estado cambia a CANCELADA	Mensaje de cancelación y estado actualizado	✓
PT-05	Ver historial de reservas del socio	Tabla con todas las reservas	Tabla correcta con estados y fechas	✓
PT-06	Acceso a /admin con rol SOCIO	HTTP 403 Forbidden	Pagina de error 403	✓
PT-07	Ranking de clases (consulta no trivial)	Lista ordenada por reservas desc.	Yoga 2, Zumba 2, HIIT 1	✓
PT-08	Informes y estadísticas del admin	Panel con ranking y ocupación media	Datos correctos y exportación disponible	✓
PT-09	Dos usuarios simultaneos	Sesiones independientes sin mezcla	Admin e incognito con datos distintos	✓
PT-10	Panel del instructor con sus clases	Lista de clases asignadas	Clase de Musculación asignada a Carlos	✓
PT-11	Exportar historial en PDF	Archivo PDF con tabla de reservas	PDF generado con cabecera PurpleForce Gym	✓
PT-12	Exportar ranking en CSV	Archivo CSV con posición y reservas	CSV con 3 filas: Yoga, HIIT, Zumba	✓

9.3 Evidencias de pruebas

PT-01 — Login correcto → redirección al dashboard

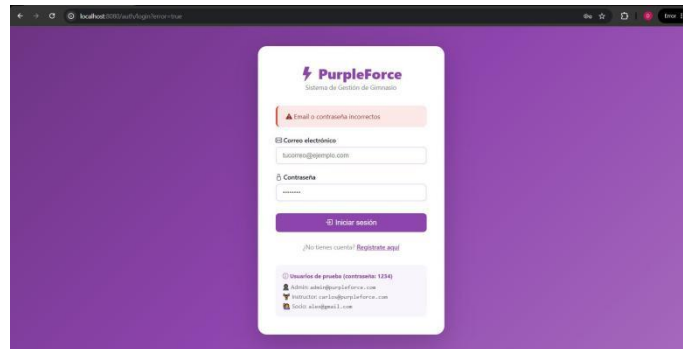
Se introduce admin@purpleforce.com y contraseña 1234. El sistema autentica y redirige al panel de administración.



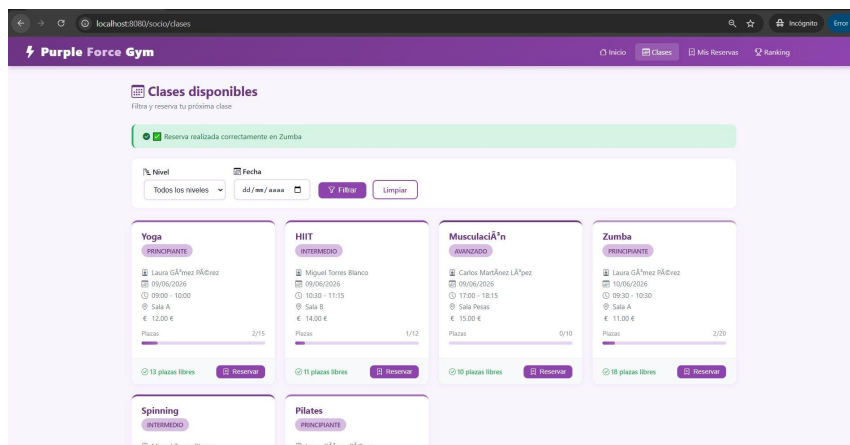
Evidencia PT-01: dashboard de administración tras login correcto.

PT-02 — Login incorrecto → mensaje de error

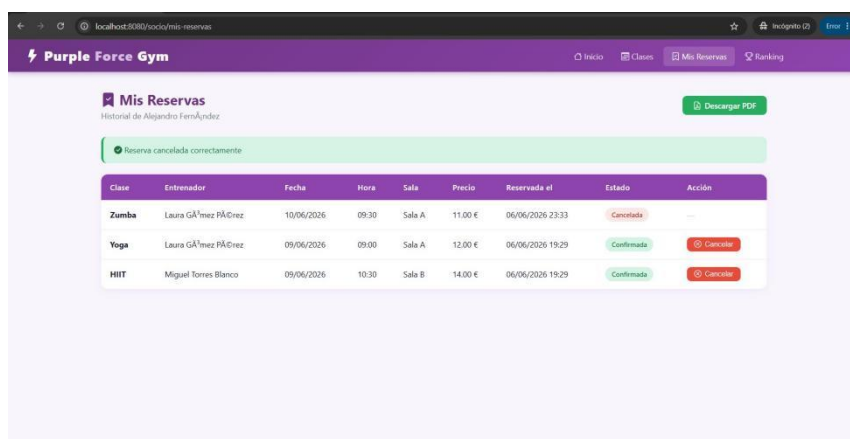
Se introduce una contraseña incorrecta. El sistema muestra el mensaje de error en rojo.



Evidencia PT-02: mensaje 'Email o contraseña incorrectos' con credenciales erróneas.

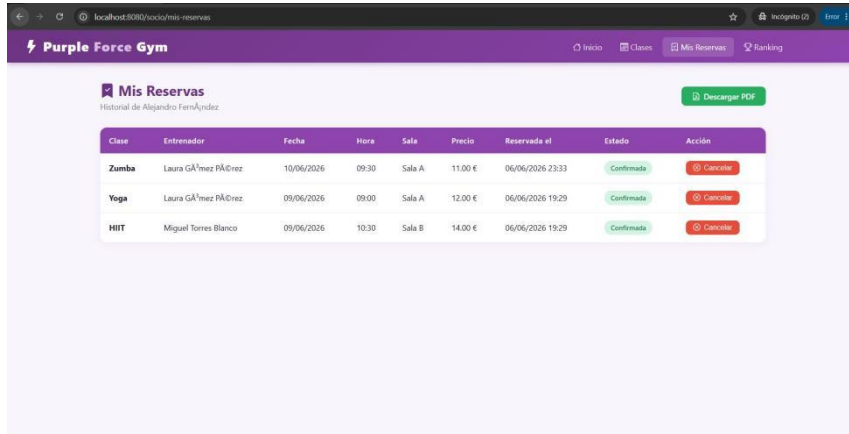
PT-03 — Reservar clase con plazas disponibles → CONFIRMADA

Evidencia PT-03: mensaje verde de reserva realizada correctamente en Zumba.

PT-04 — Cancelar reserva confirmada → estado CANCELADA

Evidencia PT-04: mensaje de cancelación y estado Cancelada en la tabla.

PT-05 — Historial de reservas del socio



Clase	Entrenador	Fecha	Hora	Sala	Precio	Reservada el	Estado	Acción
Zumba	Laura Gálvez PáDrez	10/06/2026	09:30	Sala A	11.00 €	06/06/2026 23:33	Confirmada	Cancelar
Yoga	Laura Gálvez PáDrez	09/06/2026	09:00	Sala A	12.00 €	06/06/2026 19:29	Confirmada	Cancelar
HIIT	Miguel Torres Blanco	09/06/2026	10:30	Sala B	14.00 €	06/06/2026 19:29	Confirmada	Cancelar

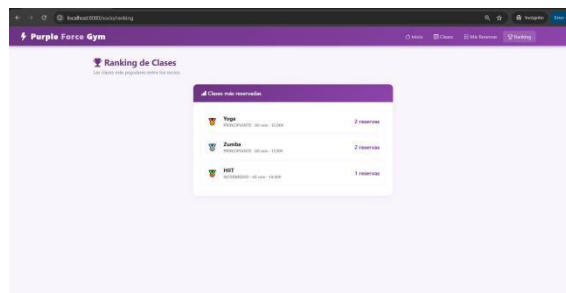
Evidencia PT-05: tabla de reservas con estados y botón Descargar PDF.

PT-06 — Acceso denegado a admin por socio → HTTP 403



Evidencia PT-06: error 403 Forbidden al acceder a /admin/dashboard con rol SOCIO.

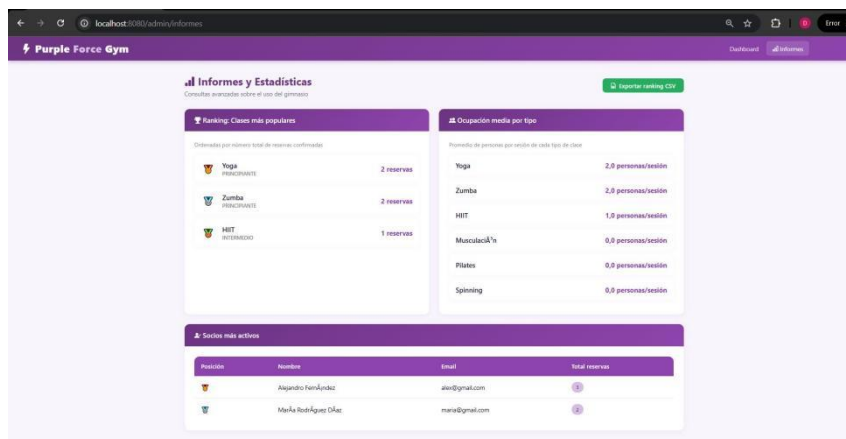
PT-07 — Ranking de clases (consulta no trivial)



Clases más reservadas	
Yoga	2 reservas
Zumba	2 reservas
HIIT	1 reserva

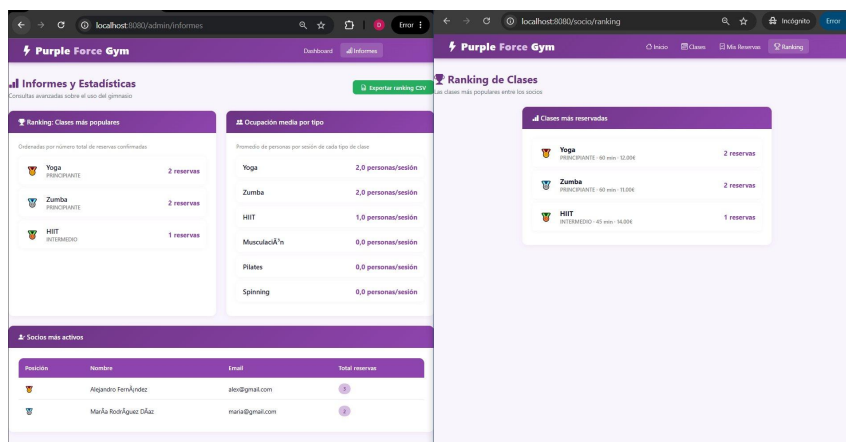
Evidencia PT-07: ranking de clases ordenadas por número de reservas.

PT-08 — Informes y estadísticas del admin (consulta no trivial)



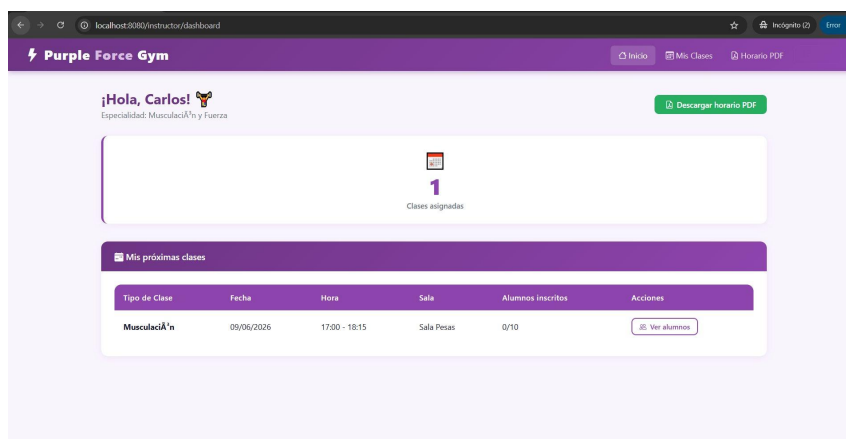
Evidencia PT-08: panel de informes con ranking, ocupación media y socios más activos.

PT-09 — Concurrencia: dos usuarios simultáneos



Evidencia PT-09: sesiones de Admin y Socio activas simultáneamente en ventanas distintas.

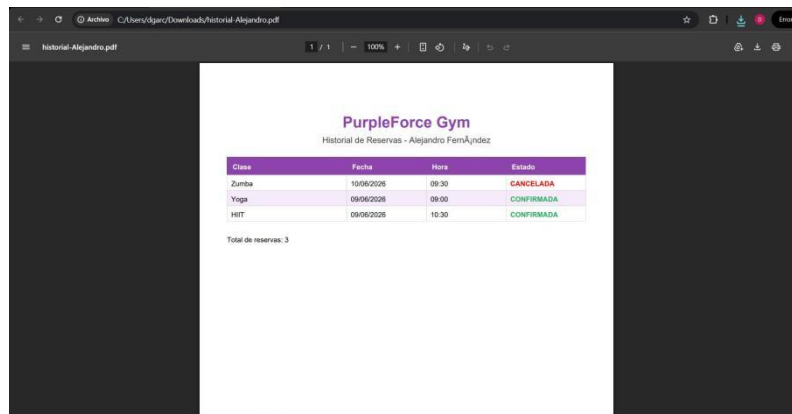
PT-10 — Panel del instructor



Evidencia PT-10: panel del instructor con clases asignadas y boton de horario PDF.

PT-11 — Exportación de historial en PDF

El socio pulsa Descargar PDF en Mis Reservas. El sistema genera un PDF con la tabla de reservas, cabecera de PurpleForce Gym y estados CONFIRMADA/CANCELADA en color.

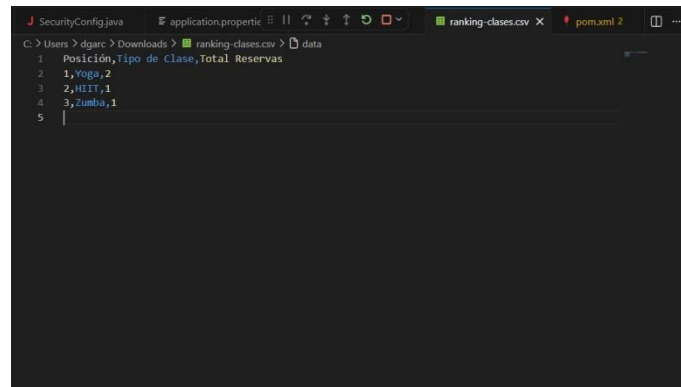


Clase	Fecha	Hora	Estado
Zumba	09/06/2025	09:30	CANCELADA
Yoga	09/06/2025	08:00	CONFIRMADA
HIIT	09/06/2025	10:30	CONFIRMADA

Evidencia PT-11: PDF generado con el historial de reservas de Alejandro Fernández.

PT-12 — Exportación del ranking en CSV

El administrador pulsa Exportar ranking CSV en el panel de Informes. El sistema genera un fichero CSV con la posición, tipo de clase y total de reservas.



```
SecurityConfig.java application.properties ranking-clases.csv pom.xml 2
C:\Users\dpere>Downloads> ranking-clases.csv > data
1 Posición,Tipo de Clase>Total Reservas
2 1,Yoga,2
3 2,HIIT,1
4 3,Zumba,1
5
```

Evidencia PT-12: fichero ranking-clases.csv abierto en VS Code mostrando los datos exportados.

10. Resultados, conclusiones y vías futuras

10.0 Métricas del proyecto

Elemento	Cantidad
Entidades JPA (modelos)	6
Controladores Spring MVC	4
Servicios	5
Repositorios JPA	5
Plantillas Thymeleaf	15+
Pruebas funcionales documentadas	12

Roles de usuario	3
Tablas en la base de datos	6
Consultas JPQL no triviales	2
Formatos de exportación	2 (PDF y CSV)

10.1 Objetivos alcanzados

- ✓ Autenticación segura con tres roles diferenciados (ADMIN, INSTRUCTOR, SOCIO).
- ✓ CRUD completo de clases, entrenadores, tipos de clase, ejercicios y usuarios.
- ✓ Sistema de reservas con control de aforo en tiempo real y concurrencia transaccional.
- ✓ Dos consultas JPQL no triviales: ranking de popularidad e historial por usuario.
- ✓ Exportación de informes en PDF y CSV.
- ✓ 12 pruebas funcionales con evidencias visuales documentadas.
- ✓ Interfaz responsiva con tema morado/blanco/negro en Thymeleaf.
- ✓ README completo con instrucciones de instalación en GitHub.
- — No implementado: gestión de pagos (descartado por alcance del proyecto).
- — No implementado: despliegue en nube (se entrega para ejecución local).

10.1b Lecciones aprendidas

- **Spring Security 6:** aprender el nuevo modelo lambda-based de configuración fue uno de los mayores retos. La separación de PasswordEncoder en una clase propia para evitar referencias circulares fue una lección clave sobre el ciclo de vida de los beans en Spring.
- **Transacciones y concurrencia:** entender como @Transactional garantiza atomicidad en escenarios de reserva simultanea ha sido la parte técnica mas valiosa del proyecto. La diferencia entre leer datos sucios y datos comprometidos (aislamiento) es algo que solo se comprende bien aplicándolo en un caso real.
- **Thymeleaf y MVC:** trabajar con un motor de plantillas del lado del servidor en lugar de una SPA (Single Page Application) ha demostrado que para proyectos de este tipo es mas sencillo, mas rápido de desarrollar y mas fácil de mantener.
- **Spring Data JPA y JPQL:** la capacidad de escribir consultas orientadas a objetos (JPQL) en lugar de SQL puro, y obtener resultados directamente como entidades Java, ha simplificado enormemente la capa de persistencia y ha reducido el código repetitivo.

10.2 Conclusiones

El desarrollo de PurpleForce Gym ha permitido consolidar y aplicar de forma práctica los conocimientos adquiridos durante el ciclo de 2º DAM: bases de datos relacionales, programación orientada a objetos con Java, desarrollo web con Spring Boot y seguridad de aplicaciones.

Los aspectos técnicos más relevantes aprendidos han sido el funcionamiento interno de Spring Security 6 con su nuevo modelo lambda, el control de concurrencia mediante transacciones JPA y la generación programática de documentos PDF con OpenPDF.

10.3 Vías futuras / mejoras

- Integración con pasarela de pago (Stripe o PayPal) para suscripciones y clases sueltas.
- Notificaciones por email automáticas al confirmar o cancelar una reserva (Spring Mail).
- API REST completa para una posible aplicación móvil (Android/iOS).
- Panel de estadísticas avanzado con gráficas de ocupación semanal (Chart.js).
- Despliegue en la nube (Railway o Render) con variables de entorno.
- Sistema de valoraciones para que los socios puntúen las clases asistidas.

11. Glosario

- **BCrypt**: algoritmo de hashing para contraseñas con salt aleatorio, resistente a fuerza bruta.
- **CRUD**: Create, Read, Update, Delete — operaciones básicas sobre datos persistentes.
- **JPA**: Java Persistence API — especificación para mapear objetos Java a tablas de BD relacional.
- **JPQL**: lenguaje de consultas de JPA orientado a objetos, similar a SQL pero sobre entidades.
- **MVC**: Model-View-Controller — patrón que separa datos, presentación y flujo de control.
- **Spring Boot**: framework Java que simplifica la configuración y arranque de aplicaciones Spring.
- **Spring Security**: módulo para autenticación, autorización y protección contra ataques web.
- **Thymeleaf**: motor de plantillas Java para generar HTML en servidor con integración Spring MVC.
- **Transacción ACID**: operación de BD que garantiza Atomicidad, Consistencia, Aislamiento y Durabilidad.

12. Bibliografía y webgrafía

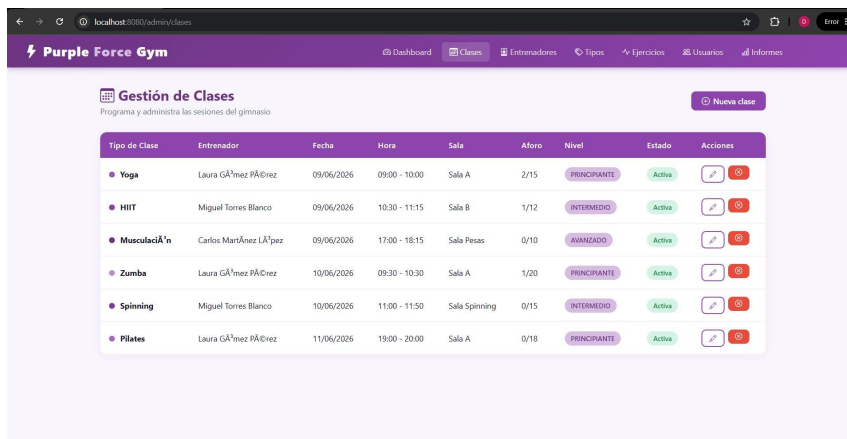
- Spring Boot Documentation (3.2): <https://docs.spring.io/spring-boot/docs/3.2.x/reference/html/>
- Spring Security Reference (6.x): <https://docs.spring.io/spring-security/reference/>
- Spring Data JPA Reference: <https://docs.spring.io/spring-data/jpa/docs/current/reference/html/>

- Thymeleaf Documentation: <https://www.thymeleaf.org/doc/tutorials/3.1/usingthymeleaf.html>
- OpenPDF GitHub: <https://github.com/LibrePDF/OpenPDF>
- Apache Commons CSV: <https://commons.apache.org/proper/commons-csv/>
- MySQL 8.0 Reference Manual: <https://dev.mysql.com/doc/refman/8.0/en/>
- Bootstrap Icons: <https://icons.getbootstrap.com/>
- Baeldung Spring Security: <https://www.baeldung.com/security-spring>
- JUnit 5 User Guide: <https://junit.org/junit5/docs/current/user-guide/>

Anexos

Anexo A — Capturas adicionales de la aplicación

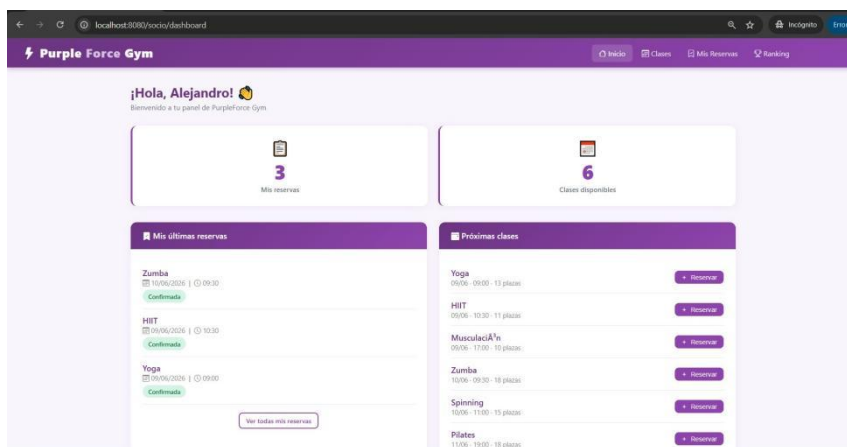
Se incluyen en este anexo capturas complementarias que muestran el funcionamiento completo de PurpleForce Gym.



The screenshot shows the 'Gestión de Clases' (Class Management) page in the Purple Force Gym admin interface. It features a table with columns for Class Type, Trainer, Date, Time, Room, Capacity, Level, Status, and Actions. A 'Nueva clase' (New class) button is in the top right.

Tipo de Clase	Entrenador	Fecha	Hora	Sala	Aforo	Nivel	Estado	Acciones
Yoga	Laura GÁmez PÁÑez	09/06/2026	09:00 - 10:00	Sala A	2/15	PRINCIPIANTE	Activa	
HIIT	Miguel Torres Blanco	09/06/2026	10:30 - 11:15	Sala B	1/12	INTERMEDIO	Activa	
Musculaci3n	Carlos Mart3nez L3pez	09/06/2026	17:00 - 18:15	Sala Pesas	0/10	AVANZADO	Activa	
Zumba	Laura GÁmez PÁÑez	10/06/2026	09:30 - 10:30	Sala A	1/20	PRINCIPIANTE	Activa	
Spinning	Miguel Torres Blanco	10/06/2026	11:00 - 11:50	Sala Spinning	0/15	INTERMEDIO	Activa	
Pilates	Laura GÁmez PÁÑez	11/06/2026	19:00 - 20:00	Sala A	0/18	PRINCIPIANTE	Activa	

Anexo A.1: gestión de clases desde el panel de administración.



The screenshot shows the user dashboard for 'Alejandro!'. It displays '3 Mis reservas' (My reservations) and '6 Clases disponibles' (Available classes). Below these are sections for 'Mis últimas reservas' (My last reservations) and 'Pr3ximas clases' (Next classes).

Clase	Fecha	Hora	Estado	Acción
Zumba	10/06/2026	09:30	Confirmada	
HIIT	10/06/2026	10:30	Confirmada	
Yoga	10/06/2026	09:00	Confirmada	

Clase	Fecha	Hora	Plazas	Acción
Yoga	09/06	09:00 - 10:00	13 plazas	Reservar
HIIT	09/06	10:30 - 11:15	11 plazas	Reservar
Musculaci3n	09/06	17:00 - 18:15	10 plazas	Reservar
Zumba	10/06	09:30 - 10:30	18 plazas	Reservar
Spinning	10/06	11:00 - 11:50	15 plazas	Reservar
Pilates	11/06	19:00 - 20:00	18 plazas	Reservar

Anexo A.2: dashboard del socio con sus ultimas reservas confirmadas.

Anexo B — SQL de ejemplo (data.sql)

El fichero data.sql carga automáticamente los datos de ejemplo al arrancar la aplicación. Incluye 3 entrenadores, 6 tipos de clase, 8 ejercicios, 6 sesiones programadas y reservas de ejemplo. Todas las sentencias INSERT usan WHERE NOT EXISTS para ser idempotentes en reinicios sucesivos.

El fichero SQL completo con la estructura de la base de datos (Gimnasio_tfc.sql) esta disponible en la carpeta /docs del repositorio: <https://github.com/dagardoc/PurpleForce>

Anexo C — Configuración de la aplicación

El fichero application.properties contiene la configuración completa del sistema.

El repositorio incluye un archivo README.md con instrucciones resumidas de puesta en marcha, la URL de clonado, los usuarios de prueba y la forma de cargar los datos de ejemplo.

Las variables mas relevantes son:

- spring.datasource.url — URL de conexion a MySQL
- spring.datasource.username / password — Credenciales (externalizadas con variables de entorno)
- spring.jpa.hibernate.ddl-auto=update — Actualiza el esquema sin borrar datos
- spring.sql.init.mode=always — Ejecuta data.sql en cada arranque
- server.port=8080 — Puerto de escucha del servidor Tomcat embebido